Chapter 3

Initial conditions

īn nīz bogzarad
This too shall pass.

Solomon – Israel Folklore Archive #126

Any simulation requires a starting point. We call these the initial conditions. Some initial conditions are simple, some are extraordinary complex. The simplest ones are often the most interesting, and lead to the clearest understanding of the complex phenomena that follow through calculation. Many final conditions eventually turn into initial conditions for future simulations. In astrophysics, the ultimate initial conditions are probably the beginning of time and the Universe, whatever they were.

3.1 Starting simulations

The generation of initial conditions for complex multiphysics simulations is a critical step in ensuring accurate and efficient computational modeling. The complexity grows with the demands and realism in the simulations. In multiphysics simulations, where multiple physical phenomena interact simultaneously, the choice of initial conditions have an unprecedented impact the simulation's stability, convergence, and the interpretebility of the results.

The challenge lies in creating a initial conditions that are consistent with the governing equations, boundary conditions, and physical constraints of the system, while, when evolved in time, teach you something about the underlying processes that govern the system's evolution.

3.1.1 Warning on initial conditions

Every simulation in AMUSE requires initial conditions to start a calculation. Typically these entail choosing dynamical quantities such as particle masses, positions, and velocities, stellar properties such as radii and luminosities, or gas properties such as density, temperature, and composition. These quantities are often the result of some prior calculation, or represent a restart from a previously stored state. In principle, any mathematically consistent set of initial choices is a valid way to start a simulation.

In practice, however, we generally want to use some reasonable, physically motivated starting point for further study.

Some initial conditions are realistic, but most are not—and most of the time that is okay. Realistic initial conditions are the goal of (almost) all researchers, but they are hard to find and generally even harder to explain. Most studies start with somewhat unrealistic initial conditions, with the expectation (or at least the hope) that they capture the fundamental physics necessary to study the desired phenomenon, and that the details of the simulation will rapidly drive the system into some physically more meaningful configuration. In follow-up studies, we can then improve on the earlier choice of initial conditions, say by covering a larger part of parameter space or by zooming in on a region of interest. In some cases, for example the stellar density at the moment of core collapse in an idealized equal-mass star cluster (see Section 4.1.3.2), the evolution is quite insensitive to the details of the initial conditions. In others, however, such as star formation in an interstellar molecular cloud (Chapter 6), the outcome depends crucially on the initial state.

A simple example of unrealistic but widely accepted initial conditions are those used in gravitational N-body simulations of star clusters. Such simulations generally start by setting the number of stars N, and distributing them in space with specified radial (spherically symmetric) density and (truncated Maxwellian) velocity profiles. Most modern studies of star clusters incorporate some sort of (parametrized) stellar evolution, so the initial conditions must then also include stellar masses and radii. The entire system is then scaled (usually by adjusting the velocities) to virial equilibrium (see Section 4.1.2), before high-precision N-body integrators are used to study the dynamical evolution of the system. From an astronomical point of view, these initial conditions are not correct, but they can still be very useful in the study of gravitational dynamics.

More realistic initial conditions for this problem might start with an interstellar gas cloud, which collapses and fragments to form stars (see Section 6.7.4). Once massive stars begin to shine, they produce winds and radiation that cause the remaining gas to be blown away, terminating further star formation and locally leaving behind a star cluster. In this case, no ad hoc specifications of the positions and masses of cluster stars are required. But of course, this only shifts the problem back in time, as the initial conditions for the gas simulation now must specify the (clumpy) density distribution and the (turbulent) motion of the cloud. Those "initial conditions" in turn depend on the large-scale flow of gas in the galaxy, and so on. The fact is that there are no "true" initial conditions for this problem, just convenient milestones in a continuous process. Most generally adopted initial conditions have problems, but doing much better can turns garbage into science, and science into garbage.

AMUSE contains an extensive (and growing) collection of codes to generate initial conditions. Very often one code provides the conditions from which another code continues. For example, when two stars collide, a stellar evolution code might provide the initial density and temperature profiles for the two colliding stars, while a gravitational dynamics code provides their relative positions and velocities. In this way two codes are used to generate the conditions for starting a third code that simulates the hydrodynamics of the collision. In the next subsections we discuss some common initial

conditions used in astrophysical studies, but we focus on the simplest ones, those that are generated by a single rather simple routine.

3.1.2 Variations in starting points

One can imagine several approaches in the generation of initial conditions. A new starting point might be generated from a single free parameter, or it may be the result of a large-scale simulation. Both are found in astronomy; people still start by randomly selecting stellar masses from a power-law mass function (which has 3 degrees of freedom), while others take the end result of a large-scale cosmological simulation to extract a specific aspect which subsequently is modeled in higher resolution or by including different physical processes.

The few-parameter generated approach results in a clean experiment, with the advantage of the easier interpretation, whereas the latter results in more complex relation between initial conditions and eventual simulation results. Which is preferable depends on your question.

We can separate the generation of initial conditions naively in the following categories (in order of increasing complexity):

- In steady-state initialization, we solve a stationary version of the problem to obtain a consistent initial state. Although it's not always clear exactly what consistent means, in our case, it would reflect the masses, positions, and velocities of objects in space, such as a solar system, or as we demonstrate in Section 4.3.1 by generating a cluster of stars, or by realizing a molecular cloud that collapses under its own gravity (Section 6.5).

- By gradually introducing loads and boundary conditions over time we assist a system to transition from simple initial values to more complex states. This ramping techniques is often used when converting the end conditions from one physical domain to start a simulation in another. An example is the conversion of a thermal-equilibrium stellar evolution model to a dynamical equilibrium hydrodynamical model of the same star, as we demonstrate in Section 6.4.2 by simulating the collision between two stars.

- Experimental data or results from simplified models can provide interesting initial conditions. This data-driven approach has become more popular with the availability of large quantities of multi-messenger data. We present a brief data-driven example in Section 9.1.4, where we generate initial conditions by mimicking observations of a giant molecular cloud.

- In multi-step initialization we use a series of simulations to progressively work up to the full complexity of the multi-physics problem. An example, provided in Section 8.7.3, explains how to simulate the evolution of planetary systems in a stellar cluster. In Section 10.3.3 (see also Figure 6.10) we go a step further by exploring the hydrodynamical collapse of a giant molecular cloud that forms stars with circumstellar disks. The disks, irradiated by the neighboring stars,

subsequently collapse and form planetary systems that evolve together with the dynamical evolution of the cluster.

The last conditions discussed in this itemized list are the most complex, and often require a trial series of initial conditions and more elaborate simulations in order to execute the eventual calculation. One can hardly talk about "initial conditions" in those cases, as they represent a morphable collection of data representing the system at an earlier epoch rather than a fixed starting point in time.

In this chapter we elaborate a little on the simplest concepts of generating initial conditions. Along the sojourn through this book, you will find many other examples of the generation of starting conditions for simulating astronomical phenomena.

3.2 Creating particle distributions in space

To generate a particle distribution, we will need to access the various functionalities of the particle set. In Section 2.2.1 we discussed the fundamental characteristics of this data structure. Here we illustrate some of its important points, such as creating and storing point-mass information for a collection of particles in space.

3.2.1 Initial condition for gravitational dynamics

For large systems, initial conditions are generally not specified exactly, and we must start with a random realization of some distribution function. In self-gravitating simulations of star clusters, the Plummer (1911) model is often adopted as an initial density profile. In AMUSE, a particle set representing a Plummer sphere of `number_of_stars` stars can be generated with

```
particles = new_plummer_model(number_of_stars)
```

The result is a `Particles` object, which contains the identities, masses, positions and velocities of the `number_of_stars` particles generated. You could inspect the content of `particles` at this point by adding an additional line

```
print(particles)
```

This would result in something like (do absolutely try this yourself!):

```
      key      mass    radius          vx  ...         x          y          z
        -      mass    length length/time  ...    length     length     length
=========  ========  ========  ==========  ... =========  =========  =========
120995...  1.00e-02  0.00e+00   7.666e-01  ...  1.95e-01  -2.01e-01   7.47e-02
 88511...  1.00e-02  0.00e+00   1.219e-01  ... -1.65e+00  -1.55e+00  -2.92e+00
 19874...  1.00e-02  0.00e+00  -2.131e-01  ...  3.08e-02   1.20e+00  -1.18e+00
      ...       ...       ...         ...  ...       ...        ...        ...
 18055...  1.00e-02  0.00e+00  -3.412e-01  ... -1.90e+00   2.35e-01   2.04e+00
 46733...  1.00e-02  0.00e+00   6.214e-01  ... -2.43e-01  -1.97e-02  -1.61e-01
 67731...  1.00e-02  0.00e+00  -1.089e-02  ... -8.33e-01   7.62e-01   2.43e+00
=========  ========  ========  ==========  ... =========  =========  =========
```

Note that the output format from your code will look somewhat different, as we have abbreviated the output due to page limitations (and we did not specify the seed in the random number generator).

As noted in §2.2.1, the `key` is a unique identifier for each particle, followed by the mass, radius, velocity, and position. Except for the first (`key`) entry, these lines may appear in any order, and the columns also may appear in a different order. The first line identifies the parameter, while the second gives the unit. In this example the units are generic (scaleless) because we have not specified any units explicitly.

The Plummer model represents a self-consistent equilibrium solution to Poisson's equation, and so is a valid description of an N-body system. But aside from the fact that it is simple and has no free dimensionless parameters in its description, it has no special claim to fame as a cluster model. Nevertheless, it is a very common choice.

```
import matplotlib.pyplot as plt
from amuse.units.nbody_system import length
from amuse.ic.plummer import new_plummer_model


def plot_plummer(number_of_particles=1000):
    fig = plt.figure(figsize=(5, 5))
    ax = fig.add_subplot(1, 1, 1)
    bodies = new_plummer_model(number_of_particles)
    ax.scatter(bodies.x.value_in(length), bodies.y.value_in(length))
    ax.set_xlim((-1, 1))
    ax.set_ylim((-1, 1))
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    plt.savefig("plummer.pdf")
    print("Saved figure in file plummer.pdf")
```

Code example 3.1: Routine to plot a Plummer sphere. Code fragment from `amuse.examples.plot_plummer`.

Some initial condition generators are not considered standard, and require a community code to be built/installed before usage. For example, for "fractal" initial conditions (Goodwin & Whitworth, 2004), we must have the `amuse.community.fractalcluster` package available. If this is not the case, using the `new_fractal_cluster_model` function will result in an error:

```
ModuleNotFoundError: No module named 'amuse.community.fractalcluster'
```

Figure 3.1 presents a small collection of various initial realizations common in N-body simulations: the Plummer sphere, a King model (with structure parameter $W_0 = 9$; King, 1966), the top view of a galactic disk plus bulge model (for which we adopted `Halogen` with parameters $\alpha = 1$, $\beta = 5$, $\gamma = 0.5$; Zemp *et al.*, 2008; Zemp, 2014), and a fractal distribution (with fractal dimension 1.6; Goodwin & Whitworth, 2004).

The various models are generated with calls similar to those discussed in Section 3.2. For the King model we write:

```
particles = new_king_model(N, W0=9.0)
```

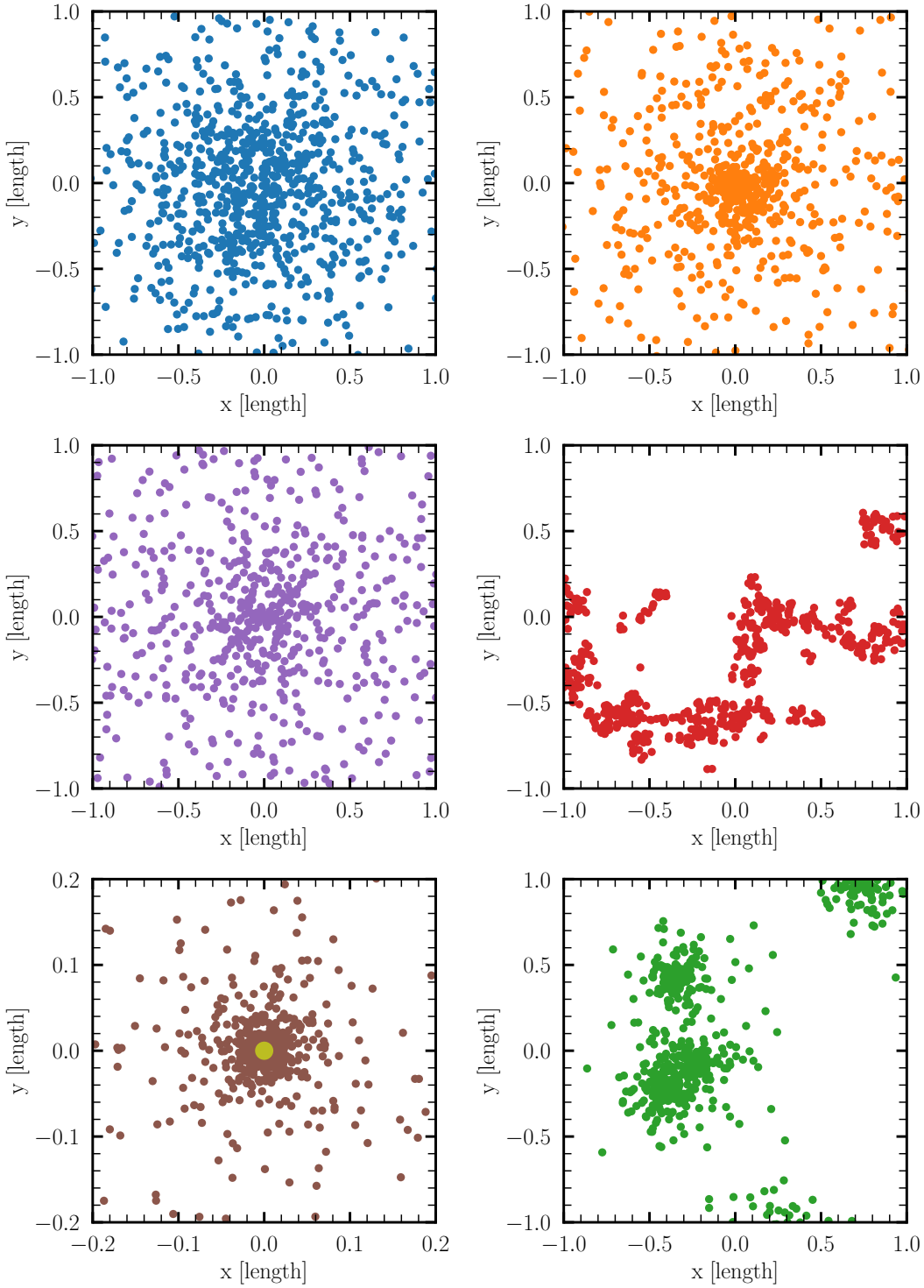

Figure 3.1: The inner parts, spanning (1 by 1 Hénon units (Hénon, 1971; Heggie & Mathieu, 1986)), of projected distributions drawn from Plummer model (top left), a King model with $W_0 = 9$ (top right), Halogen Galaxy model with $\alpha = 1$, $\beta = 5$, $\gamma = 0.5$ (middle left), a cluster with fractal dimension 1.6 (middle right), an Oort cloud with a central star (bottom left), and a hierarchical King model, each for $N = 1000$ particles. The source for making the Plummer distribution is presented in Code example 3.1. The script to generate this figure can be found in `amuse.examples.plot_density_distributions`.

The fractal distribution is generated with

```
particles = new_fractal_cluster_model(N=N, fractal_dimension=1.6)
```

For the fractal model (and for other special initial condition generators) one must include the appropriate file, which in this case can be loaded with

```
from amuse.ic.fractalcluster import new_fractal_cluster_model
```

These models have typically one free parameter. For the King model this is the dimensionless depth of the potential (W_0), while for the fractal model it is the fractal dimension (F_d).

A Galaxy disk model can be constructed with

```
particles = new_halogen_model(N, alpha=1.0, beta=5.0, gamma=0.5)
```

which has three free parameters (see Eq.1 in Zemp *et al.*, 2008, where these parameters are explained), A circumstellar disk can be generated using

```
from amuse.ext.protodisk import ProtoPlanetaryDisk
model = ProtoPlanetaryDisk(
    N,
    densitypower=1.5, Rmin=0.1, Rmax=1,
    q_out=1.0, discfraction=0.1,
).result
model.rotate(0., np.pi/6, np.pi/6)
```

This function has 10 arguments but the most important include the number of particles (N), the slope of the power-law projected density profile, the inner and outer disk radius (if defined in units these values are using the converter, see Section 2.3.1), the Toomre Q-parameter, and the mass fraction (here called `diskfraction`, indicating that the disk is 10% of the star's mass).

The best way to see what parameters a certain function in AMUSE has is by reading the actual routine. In this example, the routine lives in the `amuse.ext.protodisk` module. Every code you load has an associated source code, which you can read or copy. For this you will have to dive into the directory structure. Best is probably to take some time to make yourself familiar with AMUSEs directory structure.

The routine `ProtoPlanetaryDisk` returns a class whose parameter `result` is a particle set with additional information for performing hydrodynamical simulations, such as the internal energy, which can be used to calculate the temperature and pressure. A newly generated disk will be oriented in the x-y plane. We can rotate the disk, in azimuth (or yaw by rotation around the vertical z-axis), and elevation (or pitch, by rotation around the horizontal y axis). Here we adopted an angle of $\pi/6$ radians for both rotation angles. The result is presented in the bottom left panel in Figure 3.1.

To illustrate how we can make somewhat more complex initial conditions, we generate a composite model by first making a King model for the center of masses of the stars, and subsequently make as many King models as center of masses were generated. Here is an example of 5 King models with one-fifth the total number of stars each, as presented in the bottom right panel in Figure 3.1.

```
model_com = new_king_model(5, W)
for i in range(5):
    model = new_king_model(int(number_of_stars / 5), W)
    # Shrink the cluster.
    model.position *= 0.1
    # Keep virialized.
    model.velocity *= 3.2
    # Move to the proper coordinates.
    model.position += model_com[i].position
    model.velocity += model_com[i].velocity
```

3.2.2 Initial condition for hydrodynamics

Like particle distributions for gravitational dynamical simulations, hydrodynamical simulations also require input density and other distributions. AMUSE provides most support for smoothed particle hydrodynamics (SPH) codes (see Chapter 6), and contains a variety of distribution-generating functions. Code example 3.2 illustrates the conversion of a 1-dimensional stellar evolution model to a 3-dimensional hydrodynamical particle distribution.

```
21 def stellar_model(
22     number_of_particles=10000, mass_star=2.0 | units.MSun, time=0.0 | units.Myr
23 ):
24     star = Particle(mass=mass_star)
25     stellar_evolution = Evtwin()
26     se_star = stellar_evolution.particles.add_particle(star)
27     print(f"Evolving {star.mass} to t= {time.in_(units.Myr)}")
28     stellar_evolution.evolve_model(time)
29     print(f"Stellar type: {se_star.stellar_type}")
30     print("Creating SPH particles from the (1D) stellar evolution model")
31     sph_particles = convert_stellar_model_to_sph(
32         se_star, number_of_particles
33     ).gas_particles
34     stellar_evolution.stop()
35     return sph_particles
```

Code example 3.2: Converting a 1–D stellar evolution model to a 3–D SPH particle representation. Code fragment from `amuse.examples.plot_hydro_density_distributions`.

In Code example 3.2, in lines 25-27 the parameter `gas_particles` is returned from the function `convert_stellar_model_to_sph`. This is common practice when the called function returns a class, in this case a particle set. The return values are stored in a class containing the particle set `core_particle`, its radius (`core_radius`), and a `gas_particle` set. The resulting hydrodynamical models are plotted in Figure 3.2 (left panel); Code example 3.3 shows a portion of the plotting code.

We don't have to start from a stellar model, of course. Code example 3.4 illustrates the creation of a 3-dimensional hydrodynamical particle distribution describing a turbulent molecular cloud. As discussed in Section 3.2.1 in relation to the function `ProtoPlanetaryDisk()`, `molecular_cloud()` also returns a class whose parameter result contains the SPH particle set:

```
def plot_zams_stellar_model(number_of_particles=10000, mass_star=2.0 | units.MSun):
    length_unit = units.RSun
    sph_particles = stellar_model(number_of_particles, mass_star)
    figure = plt.figure(figsize=(6, 6))
    ax = figure.add_subplot(1, 1, 1)
    sph_particles_plot(
        sph_particles,
        min_size=500,
        max_size=500,
        alpha=0.01,
        view=(-2, 2, -2, 2) | length_unit,
    )
    ax.set_facecolor("white")
    ax.set_xlabel(f"x [{length_unit}]")
    ax.set_ylabel(f"y [{length_unit}]")

    save_file = "stellar_2MSunZAMS_projected.pdf"
    plt.savefig(save_file)
    print(f"Saved figure in file {save_file}\n")
```

Code example 3.3: Plotting the 3–D particle representation using the AMUSE `plot` module. Code fragment from `amuse.examples.plot_hydro_density_distributions`.

```
def gmc_model(
    number_of_particles=10000,
    mass_cloud=10000.0 | units.MSun,
    radius_cloud=10.0 | units.parsec,
):
    converter = nbody_system.nbody_to_si(mass_cloud, radius_cloud)
    sph_particles = molecular_cloud(
        target_number_of_particles=number_of_particles,
        convert_nbody=converter,
    ).result
    sph = Fi(converter)
    sph.gas_particles.add_particles(sph_particles)
    sph.evolve_model(1 | units.day)
    channel = sph.gas_particles.new_channel_to(sph_particles)
    channel.copy()
    sph.stop()
    return sph_particles
```

Code example 3.4: Converting a molecular cloud model to a 3–D SPH particle representation. Code fragment from `amuse.examples.plot_hydro_density_distributions`.

```
from amuse.ic.molecular_cloud import new_molecular_cloud

sph_particles = new_molecular_cloud(
    target_number_of_particles=number_of_particles, convert_nbody=converter,
)
```

Both routines can use `convert_nbody` converter to scale between dimensionless N-body units and physical units. The result is a molecular cloud represented by N particles. The particles will have all have the same mass (`mass`) and internal energy (`u`), and positions and velocities consistent with a homogeneous gas sphere with a turbulent ve-

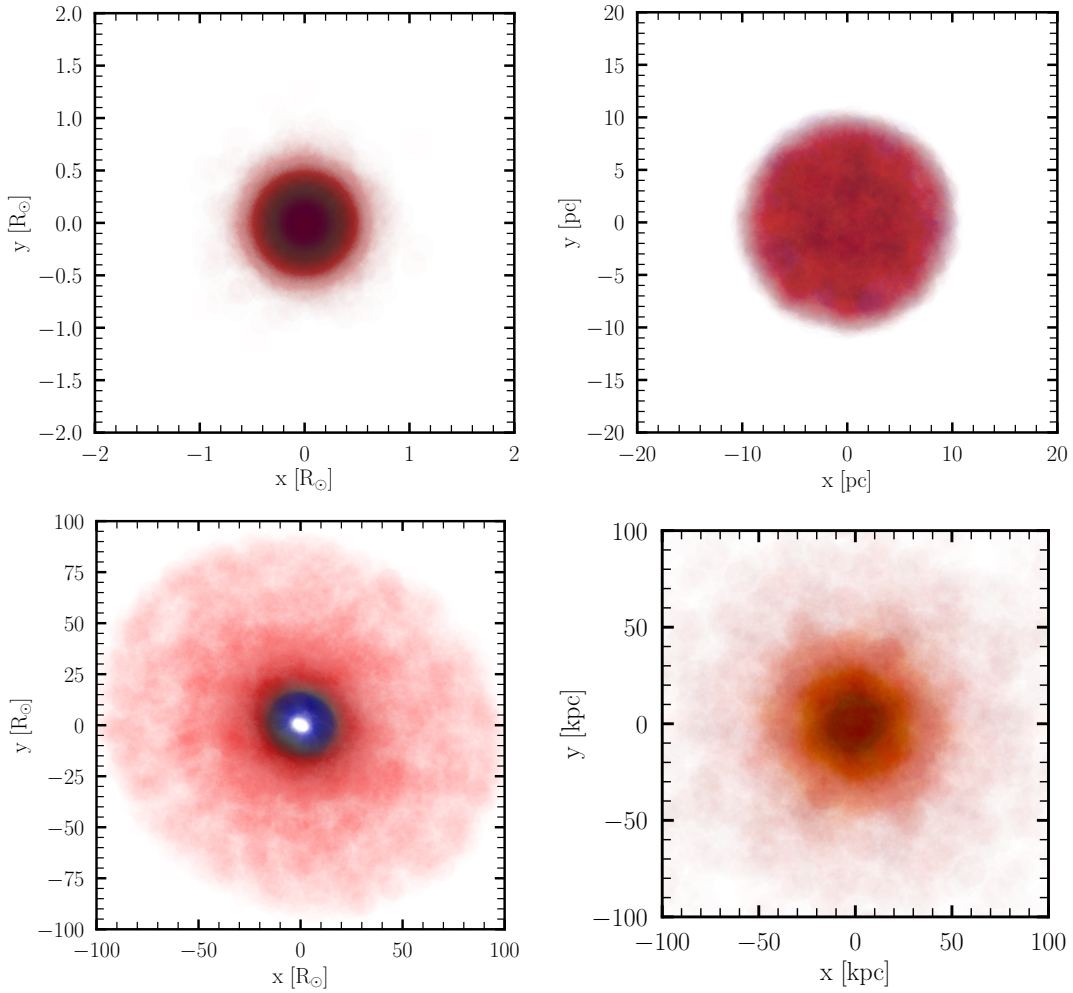

Figure 3.2: Some initial particle distributions for SPH codes. The top-row left panel shows a 2 solar mass star, the right a homogeneous spherical distribution with a turbulent velocity field, as is often used in simulations of molecular clouds. Particles are colored by their internal energy, from red (0.048 $(km/s)^2$) to blue (1.82 $(km/s)^2$). The bottom row shows a slightly tilted circumstellar disk with a total mass of 0.1% of the star's mass (left), and a $10^{10}\,M_\odot$ galaxy model. The script to generate this figure can be found in `amuse.examples.plot_hydro_density_distributions`.

locity field. (The system is integrated for a short time after creation to reduce random initial noise in the gas distribution.) The resulting density distribution is presented in Figure 3.2 (right panel). We will explore such models in more depth in Chapter 6.

We will not dwell here on more advanced code-coupling strategies, and defer discussion of radiative transport to Chapter. We postpone these advanced topics because they require multiple codes to be operational at the same time. Such coupling is discussed toward the end of this book, in Chapters 7 and 9.

3.3 Creating mass distributions

The default for `new_plummer_model` and `new_fractal_cluster_model` is to create stars of equal mass, but usually we want stars to have a range of masses, drawn from

some distribution. This distribution is known as the initial mass function, and is one of the most important quantities in stellar astrophysics.

The initial mass function is often represented as a power-law, meaning that the number of stars dN with masses between m and $m + dm$ is given by

$$\frac{dN}{dm} = A\, m^{\alpha} \tag{3.1}$$

for $M_{\rm min} < m < M_{\rm max}$. The choice $\alpha = -2.35$ (Salpeter, 1955) is so common that it is has its own standard function call in AMUSE:

```
mass_zams = new_salpeter_mass_distribution(
    number_of_stars, mass_min=mass_min, mass_max=mass_max
)
```

returns an array of `number_of_stars` masses randomly selected between `mass_min` and `mass_max`, distributed according to a Salpeter distribution, a power-law distribution with exponent $\alpha = -2.35$. The resulting mass function is presented in Figure 3.3. The naming of this mass `mass_zams` seems somewhat arcane, but refers to the mass of a star at the moment of hydrogen ignition (generally called the birth of the star) which is referred to as the zero-age main-sequence (see also Section 5.1).

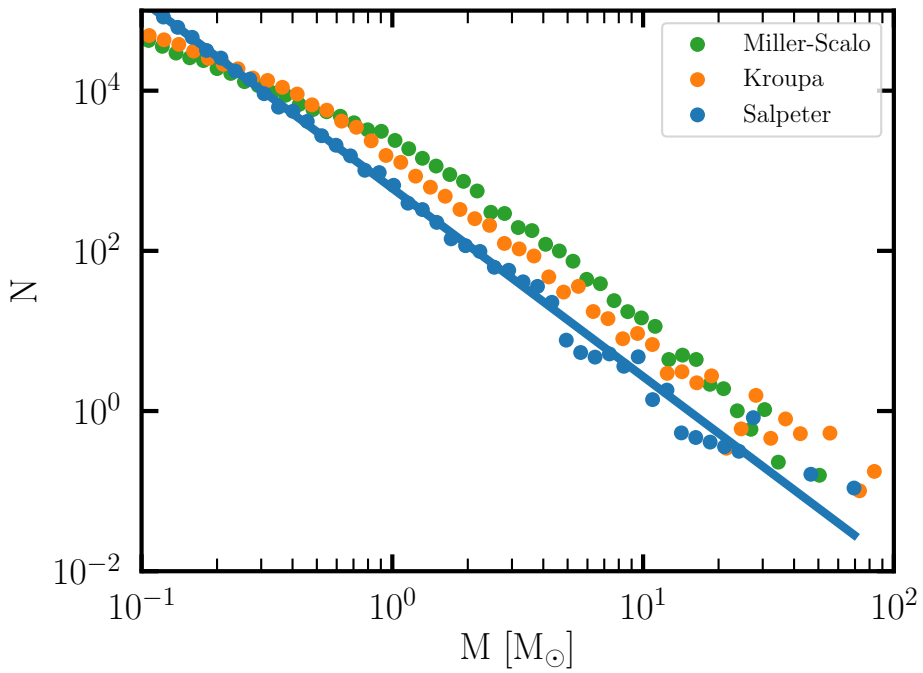

Figure 3.3: Initial mass function for 10^4 stars between 0.1 $M_\odot$ and 100 $M_\odot$. The Salpeter mass function is overplotted with an analytic power-law with a slope of -2.35 (blue). Two other often used mass functions are shows in green and orange. The latter two are described by Miller & Scalo (1979) (in green) and Kroupa (2001) (orange). An example of how this is plotted is presented in Code example 3.5. The script to generate this figure can be found in `amuse.examples.salpeter`.

```
def plot_mass_function(masses, ximf, color, label, n_bins=51):
    "Plots a histogram of the masses provided, with a power law added if ximf > 0"
    mass_min = masses.min()
    mass_max = masses.max()
    log_min = np.log10(mass_min.value_in(units.MSun))
    log_max = np.log10(mass_max.value_in(units.MSun))
    bins = np.logspace(log_min, log_max, n_bins)
    print(bins)
    bin_number, bin_edges = np.histogram(masses.value_in(units.MSun), bins=bins)
    y = bin_number / (bin_edges[1:] - bin_edges[:-1])
    x = (bin_edges[1:] + bin_edges[:-1]) / 2.0
    for yi in y:
        yi = max(yi, 1.0e-10)

    plt.scatter(x, y, s=50, c=color, lw=0, label=label)

    if ximf < 0:
        c = (
            (mass_max.value_in(units.MSun) ** (ximf + 1))
            - (mass_min.value_in(units.MSun) ** (ximf + 1))
        ) / (ximf + 1)
        plt.plot(x, len(masses) / c * (x**ximf), c=color)
```

Code example 3.5: Routine to plot the generated Salpeter mass function. Code fragment from `amuse.examples.salpeter`.

If you need a power-law mass function with a different slope, you can change the exponent by using a keyword argument, but we recommend that you use the more general version of the mass function generator:

```
mass_zams = new_powerlaw_mass_distribution(
    1000,
    mass_min=0.1 | units.MSun,
    mass_max=100 | units.MSun,
    alpha=-2.0,
)
```

which will return a list of 1000 stars with masses between 0.1 and 100 $M_\odot$, distributed according to a power law with $\alpha = -2$.

The masses can now be easily assigned to the previously generated Plummer distribution, by

```
particles.mass = mass_zams
```

or one can generate a new particle set:

```
particles = Particles(mass=mass_zams)
```

The particles can subsequently be assigned additional attributes, as already discussed. Note that changing the masses causes the system to have a different total mass and a different ratio of potential to kinetic energy, as anticipated in the function used to generate the positions and velocities, and the system generally needs to be re-scaled (see Section 4.3.1).

There are several other popular mass functions for which we generated special routines, such as Miller & Scalo (1979)

```
particles.mass = new_miller_scalo_mass_distribution()
```

and Kroupa (2001):

```
particles.mass = new_kroupa_mass_distribution()
```

A generalization of these broken power-law mass distributions is available in the class `MultiplePartIMF`, which takes arrays of breakpoints and power-law slopes allowing the user to construct a mass function of personal choice.

3.4 Creating systems of multiple stars and planets

3.4.1 The Solar system

The initial conditions of the solar system are unknown. But if we are interested in the future dynamics of the solar system we could start with the masses, positions, and velocities of the Sun and eight planets as they are observed at some moment in time, and then integrate the equations of motion forward in time. Table 3.1 presents such conditions from which such a simulation can be started.

name	mass	position			velocity		
	[M_{Jup}]	[au]			[km/s]		
Sun	1047.517	0.005717	−0.00538	-2.130×10^{-5}	0.007893	0.01189	0.0002064
Mercury	0.000174	−0.31419	0.14376	0.035135	−30.729	−41.93	−2.659
Venus	0.002564	−0.3767	0.60159	0.03930	−29.7725	−18.849	0.795
Earth	0.003185	−0.98561	0.0762	-7.847×10^{-5}	−2.927	−29.803	−0.000533
Mars	0.000338	−1.2895	−0.9199	−0.048494	14.900	−17.721	0.2979
Jupiter	1.000000	−4.9829	2.062	−0.10990	−5.158	−11.454	−0.13558
Saturn	0.299470	−2.075	8.7812	0.3273	−9.9109	−2.236	−0.2398
Uranus	0.045737	−12.0872	−14.1917	0.184214	5.1377	−4.7387	−0.06108
Neptune	0.053962	3.1652	29.54882	0.476391	−5.443317	0.61054	−0.144172

Table 3.1: A realization of the Solar System, up to the last known significant digit, calculated for 5 April 2063 at 00:00 hours—Julian date 2474649.5 days (corresponding to first contact: http://www.startrek.com/article/origin-of-first-contact-day-explained)—using the ephemerides from http://ssd.jpl.nasa.gov/txt/p_elem_t2.txt, estimating the uncertainty in the positions and velocities using the dispersion given by Folkner (2011).

The orbital parameters from this table are coded in the function `new_solar_system` (in the `amuse.ic.solarsystem` module). The parameters for moons, planetesimals, Trojans, Greeks, plutinos, the classic Kuiper belt, and the Oort cloud are not included in this table. For the moons we have a separate function that generates initial conditions for all the Solar system's moons, it is called `new_lunar_system_in_time` in the `amuse.ic.solar_system_moons` module. For the asteroids and other minor bodies, you will have to download the appropriate data, and convert the ephemerides to Cartesian coordinates using your own routine (see the assignment in Section 3.6.2).

```
12 from amuse.datamodel import Particles
13 from amuse.units import units
14
15
16 def new_sun_venus_and_earth():
17     particles = Particles(3)
18     sun = particles[0]
19     sun.name = "Sun"
20     sun.mass = 1.0 | units.MSun
21     sun.radius = 1.0 | units.RSun
22     sun.position = (855251, -804836, -3186) | units.km
23     sun.velocity = (7.893, 11.894, 0.20642) | (units.m / units.s)
24
25     venus = particles[1]
26     venus.name = "Venus"
27     venus.mass = 0.0025642 | units.MJupiter
28     venus.radius = 3026.0 | units.km
29     venus.position = (-0.3767, 0.60159, 0.03930) | units.au
30     venus.velocity = (-29.7725, -18.849, 0.795) | units.kms
31
32     earth = particles[2]
33     earth.name = "Earth"
34     earth.mass = 1.0 | units.MEarth
35     earth.radius = 1.0 | units.REarth
36     earth.position = (-0.98561, 0.0762, -7.847e-5) | units.au
37     earth.velocity = (-2.927, -29.803, -0.0005327) | units.kms
38
39     sun.color = "#FFFF00"
40     venus.color = "wheat"
41     earth.color = "deepskyblue"
42
43     particles.move_to_center()
44     return particles
```

Code example 3.6: Code to generate initial conditions for Venus and Earth orbiting the Sun. Code fragment from `amuse.examples.sun_venus_earth`.

The source code to initialize the Sun, Venus, and Earth is presented in Code example 3.6. It starts by including two basic AMUSE functionalities—particles and units:

```
10 from amuse.datamodel import Particles
11 from amuse.units import units
```

A particle set is then declared in line 14:

```
14     particles = Particles(3)
```

Attribute values are then set. The first particle in the array (`particle[0]`) is assigned to the Sun and the other two particles to Venus and Earth. (Note the details of setting three vector coordinates and units in a single line.) Then, writing

```
print(sun.position)
```

will print out the Cartesian coordinates of the Sun. After initialization, the particles are moved to the center of mass of the 3-body system, using the (class-defined) function `move_to_center()` mentioned in the previous subsection. At the end of the function, the particle set is returned to the main script with

```
34     return particles
```

A potentially easier way to achieve the initialization of the Sun, Venus and Earth, would be to use the following few lines:

```
from amuse.ic.solarsystem import new_solar_system
solar_system = new_solar_system()
SunVenusEarth = solar_system[0] + solar_system[2] + solar_system[3]
SunVenusEarth.move_to_center()
```

The third particle in this array actually represents the center of mass of the Earth-Moon system. To separate these two objects, one could use this (somewhat verbose and slow, but more precise) version:

```
from amuse.ic.solar_system_moons import new_lunar_system
solar_system = new_lunar_system(Julian_date=2474649.5 | units.day)
Sun = solar_system[solar_system.name=="sun"]
Venus = solar_system[solar_system.name=="Venus"]
Earth = solar_system[solar_system.name=="Earth"]
SunVenusEarth = Sun + Venus + Earth
SunVenusEarth.move_to_center()
```

In this latter script one actually uses the mass, position and velocity of Earth, rather than of the Earth-Moon system at Julian date 2474649.5.

The integration of the orbits can be carried out using the script in Code example 4.2 (see Section 4.4.1). Code example 3.7 presents a snippet of code to plot the planets' positions as they orbit the Sun. The results of the calculation are shown in Figure 3.4. We use `matplotlib.pyplot` (imported via convention as `plt`) to present the results, and later will show how to use an AMUSE-tailored version for plotting with units and hydrodynamical models. Note that here and throughout we provide references to the scripts that produced the figures and, for convenience, we save the plot in a file (`plt.savefile(...)`) in the current directory, and also display it on the screen (`plt.show()`).

Here we probably should explain the two function calls on lines 97 and 99, which return arrays of the requested variables (here the Cartesian coordinats along the `x` and `y` axes) for the requested particle in astronomical units. The basic functioning of this routine, as a short stand-alone snippet, might look as follows:

```
time_series = read_set_from_file("stored_particle_time_series.amuse")
bodies = time_series.history[0]
Earth = bodies[bodies.name == "Earth"]
x = get_timeline_of_attribute_as_vector(time_series, Earth, "x")
```

We do explain this in more detail in Appendix A, though.

```
80 def plot_track(particles, output_filename):
81     figure = plt.figure(figsize=(10, 10))
82     plt.rcParams.update({"font.size": 30})
83     ax = figure.add_subplot(1, 1, 1)
84     ax.minorticks_on()
85     ax.locator_params(nbins=3)
86 
87     x_label = "x [au]"
88     y_label = "y [au]"
89     plt.xlabel(x_label)
90     plt.ylabel(y_label)
91 
92     ax.scatter([0.0], [0.0], color="#CCCC00", lw=8)
93     for planet in particles:
94         if planet.name.lower() in ["venus", "earth"]:
95             ax.plot(
96                 planet.get_timeline_of_attribute_as_vector("x")[1].value_in(units.au
    ),
97                 planet.get_timeline_of_attribute_as_vector("y")[1].value_in(units.au
    ),
98                 color=planet.color,
99             )
100     ax.set_xlim(-1.3, 1.3)
101     ax.set_ylim(-1.3, 1.3)
102 
103     plt.savefig(f"{output_filename}.pdf")
104     print(f"\nSaved figure in file {output_filename}\n")
```

Code example 3.7: Plotting the orbits of Venus and Earth around the Sun using pyplot. This tells you more about `pyplot` than about AMUSE, but we present it here for completeness. Code fragment from `amuse.examples.sun_venus_earth`.

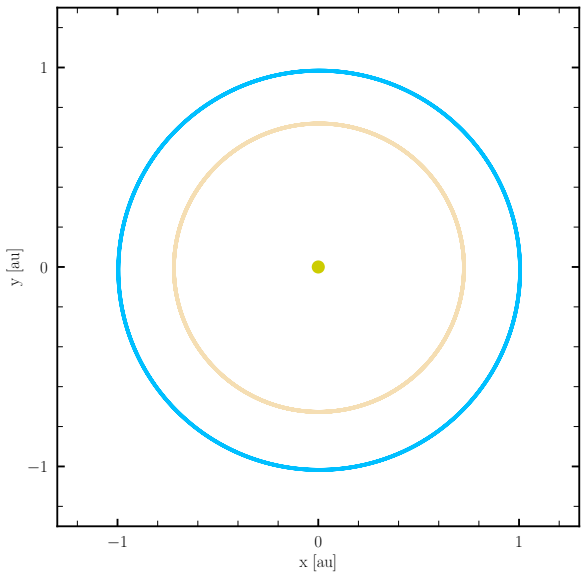

Figure 3.4: Orbital integration of the Sun, Venus and Earth for 1 yr using the script in Code example 4.2. The script to generate this figure can be found in `amuse.examples.sun_venus_earth`.

3.4.2 Other planetary systems

In principle, any planetary system can be initialized in the above described method, by providing masses, positions and velocities of the planets. Following planet formation through Oligarchic growth (Kokubo & Ida, 2002)

```
from amuse.ic import make_planets_oligarch
from amuse.units import units
mass_star = 1 | units.MSun
radius_star = 1 | units.RSun
inner_radius_disk = 10 * radius_star
outer_radius_disk = 42 | units.au
mass_disk = 0.01 * mass_star
planetary_system = make_planets_oligarch.new_system(
    mass_star,
    radius_star,
    inner_radius_disk,
    outer_radius_disk,
    mass_disk,
)
print(planetary_system.planets)
```

and to print the planetary data

```
print(planetary_system.planets[0])
```

Integrating such a system can be achieved by

```
gravity = Ph4()
gravity.particles.add_particles(planetary_system)
gravity.particles.add_particles(planetary_system.planets[0])
```

The implementation of the oligarchic planet formation model was designed by Tremaine (2015) and we adopt the implementation by Hansen & Murray (2013).

```
from amuse.ic import make_planets_oligarch
mass_star = 1 | units.MSun
radius_star = 1 | units.RSun
inner_radius_disk = 10 * radius_star
outer_radius_disk = 42 | units.au
mass_disk = 0.01*mass_star
planetary_system = make_planets_oligarch.new_system(
    mass_star,
    radius_star,
    inner_radius_disk,
    outer_radius_disk,
    mass_disk,
)
```

3.4.3 Binaries and hierarchical multiples

Generating binaries and hierarchical multiple systems may be somewhat elaborate, but if done systematically it is not hard. It is best to start with the innermost (hardest)

binary (or more than one if the system is highly hierarchical).

Let's start by making a simple binary, as in Code example 3.8. This might be a planet with moon, or a star with planet, or a galaxy with a dwarf companion. We assume that both objects are point masses, and that they have a Keplerian orbit.

```
35 def new_binary_orbit(stars, kepler):
36     """
37     Takes a Particles object of length 2 and returns a binary particle from the
38     stars, using the values from `kepler`
39     """
40     if len(stars) != 2:
41         raise ValueError("'stars' must consist of exactly two particles")
42     rel_position = as_vector_quantity(kepler.get_separation_vector())
43     rel_velocity = as_vector_quantity(kepler.get_velocity_vector())
44 
45     mu = stars[0].mass / stars.mass.sum()
46     binary = Particle()
47     binary.child1 = stars[0]
48     binary.child2 = stars[1]
49     binary.child1.position = mu * rel_position
50     binary.child2.position = -(1 - mu) * rel_position
51     binary.child1.velocity = -(1 - mu) * rel_velocity
52     binary.child2.velocity = mu * rel_velocity
53     print(binary.child1.position.in_(units.au))
54     print(binary.child2.position.in_(units.au))
55     return binary
```

Code example 3.8: Source for generating a binary of two objects with masses, positions and velocities of the two stars `star[0].mass` and `star[1].mass`. Other orbital parameters can subsequently be calculated from a Kepler solver, as initialized in Code example 3.9. Source code is available in `amuse.examples.make_binary_star`.

The Kepler solver in this routine is presented in Code example 3.9.

```
22 def new_kepler(converter):
23     """Starts and returns a new Kepler integrator with given converter"""
24     kepler = Kepler(converter)
25     kepler.initialize_code()
26     kepler.set_longitudinal_unit_vector(1.0, 0.0, 0.0)
27     kepler.set_transverse_unit_vector(0.0, 1.0, 0)
28     return kepler
```

Code example 3.9: Initializing a Kepler object for system in the x-y plane. Source code is available in `amuse.examples.make_binary_star`.

After the binary is generated, one would like to see if it really turned out as expected. A simple way to achieve this is presented in Code example 3.10, where we calculate the semi-major axis and eccentricity from the binary's Cartesian coordinates.

A higher order hierarchical multiple can be generated by iteratively using the generated binary as one of the stars in the binary generation routines.

```
62 def calculate_orbital_elements(bi, kepler):
63     """
64     Takes a binary particle and a kepler code, returns the semi-major axis and
65     eccentricity
66     """
67     comp1 = bi.child1
68     comp2 = bi.child2
69     mass = comp1.mass + comp2.mass
70     pos = comp2.position - comp1.position
71     vel = comp2.velocity - comp1.velocity
72     kepler.initialize_from_dyn(
73         mass,
74         pos[0],
75         pos[1],
76         pos[2],
77         vel[0],
78         vel[1],
79         vel[2],
80     )
81     a, e = kepler.get_elements()
82     return a, e
```

Code example 3.10: Code snippet for verifying the binary's orbital parameters. Source code is available in `amuse.examples.make_binary_star`.

3.5 Often used initial conditions

For practical purposes we have generated a series of often-used initial conditions. These are collected in a separate Python package with a number of generation functions. These functions take the form of `make_`, `add_`, `conv_` and `move_` for generating a new particle set, adding some attribute or feature, converting the particle set and moving it around. A complete list is presented in Appendix A.2.1.

These utilities can read an existing particle set from a file and write the extended or otherwise modified particle set to anotherfile, after which it can be read in by one of the other utilities. In this way it is possible to generate rather elaborate particle sets.

For example, a Plummer distribution of 100 stars in space of which 10 receive a system of 8 planets following the Oligarchic growth model on the Sun's location in the Galaxy can be realized as follows.

```
python make_Plummer_model.py -N 100 -F plummer.amuse
python name_stars.py -f plummer.amuse --name PlanetarySystem \
  --nstars 10 -F plummerNamed.amuse
python add_planet_Oligarch.py -f PlummerNamed.amuse --name PlanetarySystem \
  --Nplanets 8 -F plummerWithPlanets.amuse
python move_all_particles.py -f plummerWithPlanets.amuse --pos -8300 0.0 27 \
  --vel 11.1 240 7.25 -F plummerWithPlanetsInGalaxy.amuse
```

This procedure results in a number of files `plummer.amuse`, `plummerNamed.amuse`, `plummerWithPlanets.amuse` and the final snapshot named `plummerWithPlanetsInGalaxy`. This latter snapshot can subsequently be run with a gravity code including a Galactic potential to study the evolution of the stellar system with planets.

By combining various routines it is possible to create a broad range of particle sets that can be used as initial conditions. Of course, not all varieties of initial conditions are available, but we expect the list of initial condition routines to grow steadily with time.

3.6 Assignments

3.6.1 Create your own initial conditions generator or manipulator

The more routines we have for generating initial conditions, the more versatile AMUSE will become. For this assignment requires some idea of how these generators and manipulators are constructed. Make a generator, constructor, manipulator for an application that is not yet available in AMUSE — for example, one to generate a gaseous circumstellar disk around a single star. By the time you read this, such a routine probably has already been added to the repository, so you should find another example for which you can make a generator. Test the code, and submit it to the git repository at https://github.com/amusecode/initial-conditions-generator.

3.6.2 Solar system minor bodies

Download the ephemerides of the Solar system minor bodies (https://www.minorplanetcenter.net/iau/mpc.html), and generate a routine to convert the Kepler elements to Cartesian coordinates.

Take the initial conditions for the solar system (Table 3.1) and integrate the planets' orbits for 100 yr with the integrator of your choice (Table 4.1). Explain why you have selected this integrator, and discuss the effect of using another integrator.

1. Generate a snapshot for the Solar system at the time of first contact.

2. Check the planet positions and velocities from Table 3.1.

Bibliography

Folkner, W. M. 2011 (Oct.). Uncertainties in the JPL planetary ephemeris. *Pages 43–48 of:* Capitaine, Nicole (ed), *Journées Systèmes de Référence Spatio-temporels 2010*.

Goodwin, S. P., & Whitworth, A. P. 2004. The dynamical evolution of fractal star clusters: The survival of substructure. *A&A* , **413**(Jan.), 929–937.

Hansen, Brad M. S., & Murray, Norm. 2013. Testing in Situ Assembly with the Kepler Planet Candidate Sample. *ApJ* , **775**(1), 53.

Heggie, D. C., & Mathieu, R. D. 1986. Standardised Units and Time Scales. *Page 233 of:* Hut, Piet, & McMillan, Stephen L. W. (eds), *The Use of Supercomputers in Stellar Dynamics*, vol. 267.

Hénon, M. H. 1971. The Monte Carlo Method (Papers appear in the Proceedings of IAU Colloquium No. 10 Gravitational N-Body Problem (ed. by Myron Lecar), R. Reidel Publ. Co. , Dordrecht-Holland.). *Ap&SS* , **14**(1), 151–167.

King, Ivan R. 1966. The structure of star clusters. III. Some simple dynamical models. *AJ* , **71**(Feb.), 64.

Kokubo, Eiichiro, & Ida, Shigeru. 2002. Formation of Protoplanet Systems and Diversity of Planetary Systems. *The Astrophysical Journal*, **581**(1), 666.

Kroupa, Pavel. 2001. On the variation of the initial mass function. *MNRAS* , **322**(2), 231–246.

Miller, G. E., & Scalo, J. M. 1979. The initial mass function and stellar birthrate in the solar neighborhood. *ApJS* , **41**(Nov.), 513–547.

Plummer, H. C. 1911. On the problem of distribution in globular star clusters. *MNRAS* , **71**(Mar.), 460–470.

Salpeter, Edwin E. 1955. The Luminosity Function and Stellar Evolution. *ApJ* , **121**(Jan.), 161.

Tremaine, Scott. 2015. The Statistical Mechanics of Planet Orbits. *The Astrophysical Journal*, **807**(2), 157.

Zemp, Marcel. 2014 (July). *Halogen: Multimass spherical structure models for N-body simulations*. Astrophysics Source Code Library, record ascl:1407.020.

Zemp, Marcel, Moore, Ben, Stadel, Joachim, Carollo, C. Marcella, & Madau, Piero. 2008. Multimass spherical structure models for N-body simulations. *MNRAS* , **386**(3), 1543–1556.